



MYTHIC

ResNet-50 Retraining Guide

May 2026

Document Version 1.0
SDK Version v26.05.0

1	Introduction	3
2	Hardware Requirements	3
3	Prerequisites	3
4	Task Configuration.....	3
4.1	Dataset Setup	4
4.2	Using the standard ImageNet dataset.....	4
4.3	Using a custom dataset.....	5
4.4	DATASET environment variable	5
5	Conversion and Retraining Steps.....	5
5.1	1. Export the Pre-Trained Floating Point Model to ONNX (M1076C only).....	6
5.2	2. Convert the Floating Point model to Structural.....	6
5.3	3. Convert the Structural Model to Retraining	6
5.4	4. Retrain.....	6
5.4.1	Hyper-parameters	7
5.5	Optimize per-channel ADC ranges (optional)	7
5.6	5. Convert to Evaluation Model	7
5.7	6. Compute Accuracy Metrics	7
5.7.1	M2000	7
5.7.2	M1076C	8
5.8	7. Generate Artifact	8
5.9	8. Firmware Compilation	9
5.10	9. Benchmark AMP Inference (M1076C only).....	10

MYTHIC

1 Introduction

Mythic® provides the ResNet50 v1.5 for classification as part of its model zoo. The training environment from <https://huggingface.co/microsoft/resnet-50> is preserved in model-zoo so customers can leverage known tools. The steps to train the model and deploy to silicon are:

1. Convert a pre-trained FP32 PyTorch model to ONNX. This is used as a starting point for Mythic's analog-aware training. There are three available options:
 - The SDK includes a ResNet50 model, pre-trained on the ImageNet dataset, in ONNX format:
`resnet50_224x224_padded_7x8.onnx`
 - Alternatively, a pre-trained ResNet50 model is fetched from the Huggingface hub.
 - Alternatively, you may use your own Resnet50 (or similar classifiers like Resnet18) ONNX file.
2. Convert the FP32 model for analog-aware retraining.
3. Perform analog-aware retraining.
4. Verify the accuracy of the analog-aware model using a software simulation of the Mythic AMP.
5. Compile and deploy the model to silicon to confirm final latency and accuracy.

For customers setting up the Mythic training flow for the first time, we recommend following the steps below with the ImageNet dataset. Once all the steps have been completed with ImageNet and silicon accuracy matches expectations, switching to a custom dataset and repeating the process should be straightforward.

2 Hardware Requirements

see YOLO Retraining Guide for Mythic AMP#Hardware-Requirements

3 Prerequisites

see YOLO Retraining Guide for Mythic AMP#Prerequisites

4 Task Configuration

1. Set the environment variable `MYTHIC_SDK_ROOT` to point to the SDK directory. For example,

```
export MYTHIC_SDK_ROOT=~/.mythic_sdk/v26.05.0/
```

2. Go to the model-zoo source code directory:

```
cd $MYTHIC_SDK_ROOT/mythic-model-zoo
```

3. Activate the Python environment and the environment variables:
 - a. M1076C:

```
./scripts/huggingface_classifiers/resnet50.env
```

- b. M2000:

```
./scripts/huggingface_classifiers/resnet50-m2000.env
```

MYTHIC

The M1076C environment setup scripts will also set the environment variable `IPU_PCI_ADDRESS`. This variable defines which AMP to use. If there is only one AMP, the variable will be set correctly. If there are multiple AMPs, you must set the variable to the hexadecimal address of the AMP you want to use. To list available AMPs, you can use the command `lspci | fgrep accel`. This step can be ignored for systems that do not contain an AMP processor.

The Mythic model-zoo scripts use Hydra to handle configuration files and command-line parameters. The following sections show some basic ways of overriding configuration parameters. For advanced usage, please refer to the Hydra documentation.

4.1 Dataset Setup

The Huggingface trainer used by the Resnet50 model requires a dataset in the Apache Arrow format. This format removes serialization overhead and is very efficient. For more details, see the Huggingface docs at https://huggingface.co/docs/datasets/en/about_arrow.

4.2 Using the standard ImageNet dataset

The standard ImageNet dataset is available from the Huggingface Hub. You will need a Huggingface account and an access token, and ~400GB of free space. (160GB will be used once intermediate products are removed)

- If necessary, create a Huggingface account on <http://huggingface.co>
- If you don't have one, create an access token at <https://huggingface.co/settings/tokens>
- On the machine where you will download the dataset, install the Huggingface CLI tool:

```
pip install -U pip
pip install -U datasets "huggingface_hub[cli]"
```

- Login to Huggingface (enter your token when prompted):

```
huggingface-cli login
```

- Run Python:

```
python3
```

- In Python, execute the following:

```
from datasets import load_dataset
dataset = load_dataset("imagenet-1k") # Answer "Y" when prompted to execute
remote code
dataset.save_to_disk('imagenet_dataset') # Or provide some other location
```

- This will download the dataset and store it in the location you provided. The dataset is also saved in the Huggingface cache directories (`~/.cache/huggingface/datasets/imagenet-1k` and `~/.cache/huggingface/hub/datasets--imagenet-1k`) which you may consider cleaning to regain ~200GB of free space.

4.3 Using a custom dataset

A custom dataset may be converted to Apache Arrow format using the Huggingface API. Refer to Huggingface for full documentation: https://huggingface.co/docs/datasets/en/about_arrow .

Here is an example script for converting a dataset stored in the common ImageFolder format: A directory (DATASET_ROOT) containing two subdirectories `val` and `train` , each of which contains subdirectories corresponding to class labels (in alphanumerical order), each of which contains .JPEG images:

```
import os
import glob
from datasets import Dataset, DatasetDict, Image, load_from_disk, load_dataset
IMAGEFOLDER_DATASET_PATH = '/data/shared/global/datasets/ImageNet/ILSVRC/Data/CLS-LOC'
ARROW_DATASET_PATH = '/data/shared/global/datasets/ImageNet_arrow/'
def load_dataset_from_folder(path):
    """Processes one of train/test/val folders, loading data into a dict {'image':<path
string>, 'label':<int>}"""
    image_list = []
    label_list = []
    subfolders = sorted([folder for folder in os.listdir(path) if
os.path.isdir(os.path.join(path, folder))])
    for label, subfolder in enumerate(subfolders):
        class_path = os.path.join(path, subfolder)
        for img_path in glob.glob(class_path + '/*.JPEG'):
            image_list.append(img_path)
            label_list.append(label)
    imagedict = {'image': image_list, 'label': label_list}
    ds = Dataset.from_dict(imagedict)
    ds = ds.cast_column("image", Image())
    return ds
d_val = load_dataset_from_folder(DATASET_ROOT + '/val/')
d_train = load_dataset_from_folder(DATASET_ROOT + '/train/')
dd = DatasetDict({'train': d_train, 'validation': d_val})
dd.save_to_disk(ARROW_DATASET_PATH)
```

4.4 DATASET environment variable

Set the DATASET environment variable to point to the location of your dataset. For example,

```
export DATASET=/location/of/dataset
```

5 Conversion and Retraining Steps

Each of the following steps generates an output file. Please confirm that a valid output has been generated before proceeding to the next step.

5.1 1. Export the Pre-Trained Floating Point Model to ONNX (M1076C only)

- Skip this step if you are using your own ONNX file or the pre-trained ResNet50 model provided in the SDK.
- If you are using the ResNet50 model from the Huggingface hub, you can use the command below to fetch the model and convert it to ONNX:

```
python3 scripts/common/convert_model.py steps=to_onnx
```

- **Output:** The trained ONNX model `data/original_onnx_model/model.onnx`

5.2 2. Convert the Floating Point model to Structural

- A “structural” model is an intermediate model in the conversion process from floating point to retraining. To convert the `/path/to/the/pretrained_fp32.onnx` model to the structural model:
- If using your own ONNX file, you may need to edit the `off_chip_layers` list in `configs/huggingface_classifiers/training/training_config/resnet50_imagenet.yaml`. For M1076C, the first Conv and the final Gemm of the model must be off-chip. For M2000, only the final Gemm must be off-chip.

```
python3 scripts/common/convert_model.py steps=to_structural \
    fp_model=/path/to/the/pretrained_fp32.onnx
```

- **Output:** The structural ONNX model `data/structural.onnx`

5.3 3. Convert the Structural Model to Retraining

Conversion from structural to retraining is done with the `to_training` step:

```
python3 scripts/common/convert_model.py steps=to_training
```

- **Output:** The retraining ONNX `data/mythic.onnx`

5.4 4. Retrain

The `data/mythic.onnx` can be retrained in the `train` step.

```
python scripts/common/convert_model.py steps=train
```

- **Output:** The retrained ONNX `data/trained.onnx`

MYTHIC

5.4.1 Hyper-parameters

Hyperparameters (batch size, learning rate, etc.) can be specified in a configuration file in `configs/huggingface_classifiers/training/training_config/resnet50_imagenet.yaml`. To change parameter values, edit this file or create a new one in the same directory and put its name into the `TRAINING_SETUP` environment variable. These training parameters are passed to the Huggingface training logic.

5.5 Optimize per-channel ADC ranges (optional)

This step currently does not improve accuracy.

```
python scripts/common/convert_model.py steps=optimize_ifsr
```

- **Output:** An optimized retrained ONNX `data/optimized_ifsr.onnx`

This ONNX can be used in the accuracy evaluation steps by using `trained_model=data/optimized_ifsr.onnx`.

5.6 5. Convert to Evaluation Model

Convert the retrained `data/trained.onnx` to an evaluation model:

```
python scripts/common/convert_model.py steps=to_acm
```

- **Output:** The evaluation model `data/acm.onnx`

5.7 6. Compute Accuracy Metrics

5.7.1 M2000

Compute detailed metrics on the evaluation model with the command:

```
python3 scripts/common/convert_model.py steps=eval_trained
```

- **Output:** The accuracy metrics `data/metrics.json`.

Compute Monte Carlo-based 100PPM estimates:

```
python3 scripts/common/convert_model.py steps=mc_eval_trained  
mc_eval_trained.num_samples=100
```

- **Output:** One file per sample in `data/collected_metrics/`

MYTHIC

Print the 100PPM estimates after collecting samples:

```
python3 scripts/common/convert_model.py steps=mc_summarize_metrics
```

- **Output:** The accuracy 100PPM metrics `data/collected_metrics/metrics_summary.json`

5.7.2 M1076C

Compute detailed metrics on the evaluation model with the command:

```
python3 scripts/common/convert_model.py  
steps=[eval_fp,eval_digital,eval_signoff_v0p4,eval_signoff_v0p5]
```

- **Output:** The accuracy metrics `data/metrics.json`.

The `eval_...` steps compute accuracy metrics using different Mythic hardware models. `eval_fp` uses a floating-point model that does not apply digital quantization effects or analog noise. `eval_digital` uses a model that applies digital quantization effects but no analog noise. `eval_signoff_v0p4` and `eval_signoff_v0p5` apply digital quantization effects and an analog noise model.

- The `eval_digital` may be ~1% lower than the `eval_fp`. If it is significantly lower, silicon accuracy will be low. It is recommended that you train with a smaller learning rate.
- The `eval_signoff_v0p4` and `eval_signoff_v0p5` may be ~4% lower than the `eval_fp`. If they are significantly lower, silicon accuracy will be low. One possible cause for this is a small dataset.

5.8 7. Generate Artifact

Create an **artifact**, namely a set of files that includes the best model weights. The compiler will use the artifact to generate the firmware that is programmed into the Mythic AMP™ for inference.

To generate an artifact from the existing `data/acm.onnx` :

```
python3 scripts/common/convert_model.py steps=create_artifact \  
artifact=data/compiler_ready_artifact.tar.gz
```

- **Output:** The training artifact `data/compiler_ready_artifact.tar.gz`

Alternatively, you can override the defaults with the following parameters:

Configuration parameter or group	Description	Default value
src	An ONNX evaluation model to generate an artifact from.	<code>\${acm_model}</code> , which defaults to <code>data/acm.onnx</code> .

MYTHIC

Configuration parameter or group	Description	Default value
dest	An ONNX file to save the converted model to. A value can be a local file name or a URL of a remote file supported by https://cloudpathlib.drivendata.org/stable/ .	<code>\${env_setup.artifact_path}</code> , which defaults to <code>\${artifact}</code> , which default to <code>data/ compiler_ready_artifact.tar.g z</code>

To specify a parameter value on the command line, use `create_artifact.parameter=value` syntax, e.g., `create_artifact.dest=gs://some_location/my_artifact.tar.gz`.

5.9 8. Firmware Compilation

Once the artifact has been generated, the output can be run with the compiler to create a binary that can be loaded for inference. To generate a firmware binary with the compiler, use the `mythic-compiler` script with the compiler configuration file.

a. If using the pre-trained ResNet50 model provided in the SDK use the config: `configs/huggingface_classifiers/compiler/resnet50_imagenet_224x224x3.yaml`

b. If you are using the ResNet50 model from the Huggingface hub use the config:

`configs/huggingface_classifiers/compiler/
resnet50_imagenet_224x224x3_full_compile.yaml`

```
mythic-compiler --compiler-config configs/huggingface_classifiers/compiler/  
resnet50_imagenet_224x224x3.yaml \  
  --input-artifact data/compiler_ready_artifact.tar.gz \  
  --output-artifact FIRMWARE.tar.gz
```

Output: The firmware `FIRMWARE.tar.gz`

`mythic-compiler` takes the following parameters:

Parameter	Description
<code>--compiler-config</code>	A compiler recipe to use. The recipe for ResNet50 is <code>configs/huggingface_classifiers/compiler/resnet50_imagenet_224x224x3.yaml</code>

Parameter	Description
<code>--input-artifact</code>	A name of a training artifact generated by the steps described above.
<code>--output-artifact</code>	A name of a firmware artifact to be generated. A value can be a local file name or an URL of a remote file supported by https://cloudpathlib.drivendata.org/stable/ .

5.10 9. Benchmark AMP Inference (M1076C only)

To run a compiled model on an AMP, use the script `scripts/common/benchmark_amp_inference.py`. This script can be used to execute validation. Before running it, ensure the required environment variables are set as described in the previous sections. The inference script uses an AMP specified in the environment variable `IPU_PCI_ADDRESS`. The pre-trained model in `$MYTHIC_SDK_ROOT/models` can be used to test `benchmark_inference_amp.py` with the ImageNet dataset without training a model from scratch.

The `benchmark_amp_inference.py` script takes the following parameters:

Configuration parameter or group	Description	Default value
<code>src</code>	A firmware artifact to benchmark. A value can be a local file name or a URL of a remote file supported by https://cloudpathlib.drivendata.org/stable/ .	
<code>dataset</code>	This is only used in validation mode. A dataset to measure accuracy metrics. It should be a path to an Apache Arrow dataset.	
<code>eval_silicon.log_file</code>	A file json file to save results to.	A timestamped file <code>data/ benchmark_amp_inference_\${now:%Y-%m-%dT%H%M%S%.z}.json</code>

Validation mode is used to measure accuracy metrics for a model. The input to this script is the compiled firmware image described in the previous step. For example,

```
python3 scripts/common/benchmark_amp_inference.py src=/PATH/TO/FIRMWARE.tar.gz
```